

Checklist

Operating Platforms

1. Establish Core Operational Practices

- ☐ Treat operational work as an ongoing responsibility, not something addressed only during incidents.
- ☐ Ensure the platform team owns three core practices:
 - ☐ On-call
 - ☐ User support
 - ☐ Operational feedback
- ☐ Favor **essential practices over rigid processes**; adapt the process to the problem and team.

2. Build a Sustainable On-Call Practice

- ☐ Provide 24×7 coverage when the platform can affect critical business operations.
- ☐ Use a **merged DevOps model** where software developers and systems/operations engineers participate in the same rotation.
- ☐ Keep enough platform expertise on the rotation to diagnose issues across internal code, OSS, vendor systems, and integrations.
- ☐ Avoid relying on a separate SRE/operations team that lacks deep platform context.
- ☐ Exclude engineers from on-call only when they genuinely lack the necessary skills or willingness to operate the system.

Control On-Call Load

- ☐ Aim for each engineer to be on-call **no more than 1 week in 4**.
- ☐ Ideally, target **1 week in 6–8**.
- ☐ Track pager load and after-hours interruptions.
- ☐ Aim for **fewer than five meaningful pages per engineer per week**.
- ☐ Treat more than five business-impacting pages per week as a platform-stability problem.

- ☐ Prioritize platform stability before adding more features when operational load is excessive.

Improve Alert Quality

- ☐ Identify and eliminate false alarms.
- ☐ Review noisy alerts after deployments, batch jobs, and other predictable events.
- ☐ Keep alerts tied to meaningful customer or business impact.
- ☐ Provide other ways for engineers to see system activity without relying on pages.
- ☐ Regularly tune alert thresholds and conditions.

Secondary On-Call

- ☐ Avoid routine secondary rotations that merely forward incidents to the primary team.
- ☐ Use another platform team as secondary only for rare, true emergencies.
- ☐ Ensure secondary engineers have useful runbooks and escalation paths.

Compensation and Fairness

- ☐ Focus first on reducing unnecessary after-hours work.
- ☐ Make on-call expectations and compensation consistent and fair.
- ☐ Avoid using extra pay as a substitute for fixing an unsustainable on-call load.

3. Make Platform Engineers Participate in Support

- ☐ Ensure platform engineers regularly hear directly from users.
- ☐ Use support work to expose usability problems, confusing abstractions, and missing platform capabilities.
- ☐ Avoid shielding engineers so completely that they lose empathy for users.
- ☐ Treat repeated support questions as potential product or documentation problems.

4. Formalize Support Levels

- ☐ Categorize incoming requests before deciding staffing levels.
- ☐ Separate common request types, such as:
 - ☐ Production troubleshooting

- ☐ Noncritical platform bugs
- ☐ Help using the platform
- ☐ Feature requests submitted as bugs
- ☐ Pull-request reviews
- ☐ Complex architectural/design questions
- ☐ Investigate categories that generate more than a few requests per week.
- ☐ Reduce repeated requests through:
 - ☐ Better documentation
 - ☐ User training
 - ☐ Improved diagnostics
 - ☐ Platform/product changes
- ☐ Define clear support service levels.
- ☐ Specify what qualifies as a **critical incident**.
- ☐ Define when users may page the platform team.
- ☐ Define expected response times for noncritical issues.
- ☐ Tie urgent engagement to concrete business impact, not merely user preference.
- ☐ Build stakeholder trust so support rules are respected.

5. Separate Noncritical Support from On-Call

- ☐ Keep critical, after-hours incidents on the on-call rotation.
- ☐ Move noncritical support work to a **business-hours support rotation** when volume becomes substantial.
- ☐ Avoid assigning all support permanently to a product manager or engineering manager.
- ☐ Track total operational load across both support and on-call duties.
- ☐ Reassess staffing if operational work consumes roughly half or more of engineering capacity.

6. Add Support Specialists When Needed

- ☐ Hire a support specialist only after improving documentation, tooling, training, and platform usability.
- ☐ Prefer versatile technical support engineers who can handle both basic and advanced cases.
- ☐ Avoid creating a dead-end Tier 1 role with no growth path.
- ☐ Consider hiring engineers with nontraditional backgrounds who can grow into platform engineering.

- ☐ Give support specialists structured training and development opportunities.
- ☐ Reassess whether support staffing needs are permanent or temporary.

7. Support Far-Flung Time Zones Deliberately

- ☐ Establish reliable **business-hours support** for new offices or acquisitions in distant time zones.
- ☐ Consider temporarily relocating an experienced team member to train the new location.
- ☐ Consider a “reverse embed,” bringing someone from the new location to the main team for several months.
- ☐ Hire locally only when training and integration can be supported effectively.
- ☐ Avoid indefinite off-hours “best effort” support from the existing team.

8. Scale Support with an Engineering Support Organization

- ☐ At a large scale, consider a centralized Tier 1 engineering support organization.
- ☐ Keep platform teams responsible for Tier 2 and escalated cases.
- ☐ Define different SLAs based on application/business criticality.
- ☐ Require critical customer/application teams to maintain their own on-call coverage when appropriate.
- ☐ Maintain a healthy mix of software development and systems expertise.
- ☐ Build a network of domain experts for difficult Tier 2 cases.
- ☐ Hold regular feedback sessions between support teams and platform on-call engineers.
- ☐ Convert recurring support pain into documentation, tooling, and platform improvements.

Operational Feedback Practices

9. Define Useful SLOs and SLAs

- ☐ Create SLOs that help engineers understand platform health.
- ☐ Use SLOs to guide monitoring and on-call investigation.
- ☐ Keep the number of **customer-facing SLOs small**—ideally only a handful.

- ☐ Ensure customer-facing SLOs minimize false positives.
- ☐ Prefer an occasional explainable false negative over constant noisy alarms.
- ☐ Use more detailed internal SLOs for broad system coverage.
- ☐ Internally, do **not** optimize solely to minimize false positives.
- ☐ Investigate both false positives and false negatives.
- ☐ Use failing-minute trends to communicate operational problems when useful.
- ☐ Treat error budgets as optional rather than automatically required.
- ☐ Use error budgets only when they clearly improve decision-making and justify their overhead.

10. Implement Change Management

- ☐ Document all production changes.
- ☐ Review all production changes.
- ☐ Test changes before they handle production traffic directly.
- ☐ Maintain explicit change-management discipline until safe automation is mature.
- ☐ Invest in release engineering when the risk of manual deployment remains high.
- ☐ Use incidents caused by changes as feedback for better automation.
- ☐ Avoid removing safeguards before reliable automated alternatives exist.
- ☐ Keep release engineering proportional; do not turn the platform team into a massive deployment-tooling project.

11. Build Synthetic Monitoring

- ☐ Implement synthetic monitoring in addition to metrics, logs, and tracing.
- ☐ Exercise the platform from the user/customer perspective.
- ☐ Test important end-to-end workflows rather than only individual components.
- ☐ Ensure synthetic tests can detect failures before customers report them.
- ☐ Use synthetic monitoring to validate customer-facing SLOs.
- ☐ Include realistic multi-component and multi-API scenarios.
- ☐ Use synthetic failures to uncover confusing or flaky platform behavior.
- ☐ Let engineers gain operational experience by troubleshooting synthetic failures.
- ☐ Use synthetic tests to improve incident triangulation.
- ☐ Budget ongoing engineering and infrastructure capacity for synthetic monitoring.

12. Run Regular Operational Reviews

- ☐ Hold a simple **30–60-minute weekly operational review** at the team level.
- ☐ Review:
 - ☐ Pages and lower-severity on-call issues
 - ☐ Customer support issues
 - ☐ Incident/outage postmortems
 - ☐ Production changes
 - ☐ Important SLIs and SLOs
- ☐ Rotate meeting preparation and facilitation where practical.
- ☐ Focus on patterns, outliers, trends, and actionable problems.
- ☐ Keep the meeting lightweight but rigorous.
- ☐ Avoid turning reviews into process theater.

Organization-Level Reviews

- ☐ Review the highest-impact incidents and outages.
- ☐ Review meaningful metric outliers across teams.
- ☐ Use technically credible facilitators who can keep the discussion focused.
- ☐ Avoid one-size-fits-all operational processes across every platform team.
- ☐ Adapt actions to each platform's situation.

13. Close the Feedback Loop

- ☐ Convert operational data into engineering priorities.
- ☐ Track remediation items from postmortems until completed.
- ☐ Ensure recurring pages trigger investigation into their root causes.
- ☐ Use support trends to improve documentation, tooling, and platform design.
- ☐ Ensure engineering managers and platform leaders participate in operational reviews.
- ☐ Give leaders enough operational context to make informed investment decisions.
- ☐ Balance feature development with reliability and operational work.
- ☐ Hold platform engineers accountable for operating the systems they build.

Quick Health Check

- ☐ On-call load is sustainable.
- ☐ False alarms are uncommon.
- ☐ Engineers regularly interact with users.

- ☐ Critical and noncritical support are handled differently.
- ☐ Repeated support issues are being eliminated, not merely answered.
- ☐ Customer-facing SLOs are few and meaningful.
- ☐ Production changes are documented, reviewed, and tested.
- ☐ Synthetic monitoring covers critical end-to-end paths.
- ☐ Operational reviews happen regularly.
- ☐ Postmortem and review actions are actually completed.
- ☐ Leadership uses operational feedback to change priorities.
- ☐ Operational work is treated as a core part of platform engineering—not as an interruption to it.